

1.

ANS: There are 4 join conditions because there are 5 tables:

branch → customer → account → account_txn → card

Each connection between two tables needs a join condition.

If you provide one fewer join condition, one table is not properly connected, which can produce a Cartesian product / incorrect row multiplication.

2.

ANS: a) Returns only customers who have an account.

b) Returns all customers, including those who have no account.

LEFT JOIN returns more rows

3.

ANS: The WHERE condition removes the rows where l is NULL, so branches without overdue loans disappear.

```
SELECT b.branch_name, l.loan_id
FROM branch b
LEFT JOIN loan l
  ON l.branch_id = b.branch_id
 AND l.status = 'OVERDUE';
```

Now every branch appears, even if it has no overdue loan.

4.

ANS: General shape:

```
SELECT ... FROM table_a a
LEFT JOIN table_b b
  ON b.key = a.key
WHERE b.key IS NULL;
```

It finds rows in table_a that do not have a matching row in table_b.

You should test a non-nullable key column from table_b, usually its primary key.

Why?

Because if that column itself can contain NULL, you cannot reliably distinguish "no matching row" from "matching row whose column is NULL."

5.

ANS: A LEFT JOIN is needed because Branch Managers may not have a manager. With an INNER JOIN, employees without a matching manager would be removed. So using an inner join would lose top-level employees/Branch Managers from the staff list.

6.

ANS:

Expression — Counts NULLs? — Expected

COUNT(*) — Yes, rows are counted — 36

COUNT(interest_rate) — NO — Depends on NULL values

COUNT(DISTINCT account_type) — NO — Number of distinct non-NULL

AVG(balance) — No — Average of non-NULL balances

7.

ANS: Every selected column must either be:

included in the GROUP BY, or

used inside an aggregate function such as COUNT(), SUM(), AVG(), MIN() or MAX().

The database cannot simply pick a value for an ungrouped column because multiple different values may exist within the same group, so there would be no unambiguous value to return.

8.

ANS:

	WHERE	HAVING
--	-------	--------

Filters what?	Individual rows	Groups
---------------	-----------------	--------

Runs when?	Before GROUP BY	After GROUP BY
------------	-----------------	----------------

Can reference aggregates?	No	Yes
---------------------------	----	-----

Which should you prefer? | Use WHERE when filtering rows | Use 'HAVING'

|

9.

ANS:

```
a)SELECT COUNT(*)  
FROM customer c  
WHERE NOT EXISTS ( SELECT 1 FROM fixed_deposit fd  
WHERE fd.nominee_name = c.first_name);
```

```
b)SELECT COUNT(*)  
FROM customer c  
WHERE c.first_name NOT IN (  
SELECT nominee_name  
FROM fixed_deposit  
WHERE nominee_name IS NOT NULL);
```

10.

ANS: Fan-out = one-to-many joins multiplying rows and potentially inflating aggregates.

Name: nikhil dhalan

Emp_id-90119832-----

--Q11)

```
select c.card_type,c.card_number_masked,c.status,
       a.account_number,cu.first_name || ' ' || cu.last_name AS
customer_name,b.branch_name
from card c
join account a ON c.account_id = a.account_id
join customer cu ON a.customer_id = cu.customer_id
join branch b ON a.branch_id = b.branch_id
ORDER BY b.branch_name, customer_name;
```

--Q12)

```
select cu.first_name || ' ' || cu.last_name AS customer_name,
       a.account_number,round(a.balance,2) as The_balance,b.branch_name
from customer cu
join branch b ON b.branch_id = cu.home_branch_id
left join account a ON cu.customer_id = a.customer_id
where b.branch_name IN ('Koramangala', 'Whitefield') OR a.account_id IS NULL
ORDER BY a.balance desc nulls last;
```

--Q13)

```
select a.account_number,a.account_type,
       a.status,cu.first_name || ' ' || cu.last_name AS customer_name
from account a
join customer cu ON a.customer_id = cu.customer_id
left join account_txn t
       ON a.account_id = t.account_id
where t.txn_id IS NULL;
```

--Q14)

```
select a.account_number,a.account_type,
       cu.first_name || ' ' || cu.last_name AS customer_name
from account a
join customer cu ON a.customer_id = cu.customer_id
left join card c ON a.account_id = c.account_id
where c.account_id IS NULL
order by a.account_number;
```

--to get count

```

select COUNT(*) AS account_without_card
from account a
left join card c ON a.account_id = c.account_id
where c.account_id IS NULL;

-----

--Q15)
select e.first_name || ' ' || e.last_name as employee_name,e.designation,
       b.branch_name,
       COALESCE(m.first_name || ' ' || m.last_name, '(no Manager)') AS reports_to
from employee e
join branch b ON e.branch_id = b.branch_id
left join employee m ON e.manager_id = m.employee_id
order by b.branch_name, e.employee_id;

-----

--Q16)
select  m.first_name || ' ' || m.last_name AS manager_name,
        b.branch_name,COUNT(e.employee_id) AS team_size,
        COALESCE(SUM(e.salary), 0) AS total_team_salary
from employee m
join branch b ON m.branch_id = b.branch_id
left join employee e ON e.manager_id = m.employee_id
WHERE m.designation = 'Branch Manager'
group by m.employee_id,m.employee_name,b.branch_name
order by b.branch_name;

-----

--Q17)
--create table
CREATE TABLE loan_risk_band (
    band_code VARCHAR(4) not null PRIMARY KEY,
    band_name VARCHAR(20) not null,
    min_outstanding numeric(15,2) not null,
    max_outstanding numeric(15,2) not null
);

--insert the bands
insert into loan_risk_band
(band_code, band_name, min_outstanding, max_outstanding)
VALUES ('L1', 'Small', 0, 500000);

insert into loan_risk_band
(band_code, band_name, min_outstanding, max_outstanding)

```

```

VALUES ('L2', 'Medium', 500000, 2000000);

insert into loan_risk_band
(band_code, band_name, min_outstanding, max_outstanding)
VALUES ('L3', 'Large', 2000000, 5000000);
insert into loan_risk_band
(band_code, band_name, min_outstanding, max_outstanding)
VALUES ('L4', 'Very Large', 5000000, 99999999);

--final step
select r.band_code,r.band_name,
       COUNT(l.loan_id) AS loan_count,
       COALESCE(SUM(l.outstanding_amount), 0) AS total_outstanding
FROM loan_risk_band r
left join loan l ON l.outstanding_amount >= r.min_outstanding
       AND l.outstanding_amount <= r.max_outstanding
group by r.band_code,r.band_name
ORDER BY r.band_code;

-----
--Q18)
select b.branch_name,
       COUNT(DISTINCT c.customer_id) AS customer_count,
       COUNT(DISTINCT a.account_id) AS account_count,
       ROUND(SUM(a.balance), 2) AS total_balance,
       ROUND(AVG(a.balance), 2) AS average_balance
from branch b
left join customer c ON b.branch_id = c.home_branch_id
left join account a ON c.customer_id = a.customer_id
group by b.branch_name
ORDER BY total_balance DESC;

-----
--Q19)
select l.loan_type,
       COUNT(l.loan_id) AS loan_count,
       round(SUM(l.principal_amount),2) AS total_principal,
       round(SUM(l.outstanding_amount),2) AS total_outstanding,
       ROUND(
           SUM(l.outstanding_amount) / (SUM(l.principal_amount), 1) * 100,
           1
       ) AS outstanding_percentage

```

```

from loan l
group by l.loan_type
ORDER BY outstanding desc;

-----

--Q20)
select
    channel,
    SUM(CASE
        WHEN status = 'SUCCESS' THEN 1
        ELSE 0
    END) AS successful_transactions,

    SUM(CASE
        WHEN status = 'FAILED' THEN 1
        ELSE 0
    END) AS failed_transactions,

    SUM(CASE
        WHEN status = 'SUCCESS' THEN amount
        ELSE 0
    END) AS total_successful_value,

    ROUND(
        SUM(CASE
            WHEN status = 'FAILED' THEN 1
            ELSE 0
        END)
        / NULLIF(COUNT(*), 2) * 100,
        2
    ) AS failure_rate
FROM account_txn
GROUP BY channel
ORDER BY failure_rate DESC;

-----

--Q21)
select b.branch_name,
    COUNT(l.loan_id) AS loan_count,
    SUM(l.outstanding_amount) AS total_outstanding
from branch b
join loan l ON b.branch_id = l.branch_id

```

```

GROUP By b.branch_name
having sum(l.outstanding_amount) > 2000000
order by total_outstanding DESC;

-----

--Q22)
select
    c.first_name || ' ' || c.last_name AS customer_name,
    COUNT(a.account_id) AS account_count,
    SUM(a.balance) AS combined_balance
from customer c
join account a ON c.customer_id = a.customer_id
group by c.customer_id, c.first_name,
         c.last_name
HAVING COUNT(a.account_id) >= 2
order by combined_balance DESC;

```

```

-----

--Q23)
select
    to_char(txn_date, 'YYYY-MM') AS month,
    COUNT(*) AS transaction_count,
    SUM(CASE
        WHEN txn_type = 'CREDIT' THEN amount
        ELSE 0
    END) AS total_credits,

    SUM(CASE
        WHEN txn_type = 'DEBIT' THEN amount
        ELSE 0
    END) AS total_debits,

    SUM(CASE
        WHEN txn_type = 'CREDIT' THEN amount
        WHEN txn_type = 'DEBIT' THEN -amount
        ELSE 0
    END) AS net_movement

from account_txn
group by to_char(txn_date, 'YYYY-MM')

```

```

ORDER BY month;

-----

--Q24)
select account_number,round(balance,2) as balance
from account
where balance > 2*(
    select  AVG(balance)
    from account
)
order by balance DESC;

-----

--Q25)
select b.branch_name,
       a.balance AS largest_account_balance
from branch b
join account a ON a.branch_id = b.branch_id
where  a.balance = (
    SELECT MAX(a2.balance)
    FROM account a2
    WHERE a2.branch_id = b.branch_id
)
ORDER BY largest_account_balance desc;

-----

--Q26)
select
    c.customer_id,c.first_name || ' ' || c.last_name AS customer_name
from customer c
JOIN loan l ON l.customer_id = c.customer_id
left join fixed_deposit fd
    ON fd.customer_id = c.customer_id
where fd.customer_id IS NULL
ORDER BY c.customer_id;

-----

--Q27)
SELECT ct.customer_id, ct.customer_name, ct.total_value
FROM (SELECT c.customer_id,
             c.first_name || ' ' || c.last_name AS customer_name,
             SUM(t.amount) AS total_value
        FROM customer c
        JOIN account a ON a.customer_id = c.customer_id

```

```

        JOIN account_txn t ON t.account_id a.account_id
    WHERE t.status = 'SUCCESS'
    GROUP BY c.customer_id, c.first_name, c.alast_name) AS ct

WHERE ct.total_value > (SELECT AVG(total_value)
                        FROM (SELECT SUM(t.amount) AS total_value,
                                FROM account_txn t
                                join account a ON a.account_id=t.account_id
                                WHERE t.status SUCCESS
                                Group by a.customer_id) as per_customer)

ORDER BY ct.total_value DESC;

```

--Q28)

```

select customer_id, product_type,
       round(amount,2) AS amount
FROM (SELECT
      customer_id,
      'DEPOSIT' AS product_type,
      principal AS amount
    FROM fixed_deposit
      UNION ALL
    SELECT
      customer_id,
      'LOAN' AS product_type,
      principal_amount AS amount
    FROM loan
      UNION ALL
    select customer_id, 'ACCOUNT', balance from
    account)
ORDER BY customer_id, product_type

```

--Q29)

```

select customer_id from fixed_deposit
intersect
select customer_id from loan;

select customer_id from fixed_deposit
except
select customer_id from loan;

```

```
select customer_id from loan
except
select customer_id from fixed_deposit;

select customer_id from loan
union
select customer_id from fixed_deposit;
```


30.

ANS: SELECT

```
    b.branch_name,  
    COUNT(DISTINCT c.customer_id) AS customers,  
    COALESCE(SUM(a.balance), 0) AS total_deposits,  
    COALESCE(SUM(tx.total_transacted), 0) AS total_transacted  
FROM branch b  
LEFT JOIN customer c  
    ON c.home_branch_id = b.branch_id  
LEFT JOIN account a  
    ON a.customer_id = c.customer_id  
LEFT JOIN (SELECT account_id,  
    SUM(amount) AS total_transacted  
FROM account_txn  
WHERE txn_date BETWEEN DATE '2026-01-01'  
    AND DATE '2026-03-31'  
GROUP BY account_id)  
tx ON tx.account_id = a.account_id  
GROUP BY b.branch_id, b.branch_name;
```

31.

ANS:

GROUP BY and HAVING alone cannot find the top 3 within each branch.

First, calculate each customer's balance, then use a window function such as:

```
ROW_NUMBER() OVER (  
    PARTITION BY branch_id  
    ORDER BY balance DESC)
```

Then filter:

```
WHERE rn <= 3
```

Module: This requires the module/topic covering window/analytic functions. The supplied M03 PDF does not identify that module's number/name, so I would not invent it.

32.

ANS: NOT IN can return zero rows if the subquery contains NULL.

Always test for NULL and validate reports where zero is a possible result.

33.

ANS: Check the execution plan.

Check indexes and statistics.

Consider partitioning/performance optimization.

34.

ANS:

Use EXCEPT in both directions:

Extract EXCEPT ICICI → transactions missing in ICICI.

ICICI EXCEPT Extract → unexpected ICICI transactions.

Both empty → data matches.

A difference can also be caused by format/timestamp/rounding differences.

35.

ANS: The pack now shows branch-level customer, account and transaction information.

Finding 1: Use actual query result.

Finding 2: Use actual query result.

Limitation: Joins and aggregation can affect totals if relationships cause row multiplication.